

Documentation Technique :

Exploration Pyrénées

Tourisme Durable & Analyse Territoriale

Ce document présente l'architecture et le fonctionnement du logiciel **Exploration Pyrénées**. Il détaille les scripts, les API et les mécanismes de cartographie pour faciliter la maintenance et l'évolution du code.

1. Présentation et Objectifs

L'application web **Exploration Pyrénées - Tourisme Durable** est une plateforme cartographique interactive orientée vers l'analyse territoriale et la gestion des flux touristiques. Elle vise à recenser, catégoriser et mettre en relation des Points d'Intérêt (POI) issus d'OpenStreetMap (OSM) avec des indicateurs démographiques, d'accessibilité en transports, de capacité d'hébergement et de préservation environnementale.

2. Architecture du Projet

Le projet suit l'architecture d'une **Single Page Application (SPA)** développée en JavaScript natif (ES6 Modules) côté client, adossée à des pré-traitements asynchrones côté serveur via Node.js.

- **Couche de Présentation Cartographique** : Gérée par la bibliothèque **Leaflet (v1.9.4)** enrichie des modules `Leaflet.draw` pour la création d'emprises d'analyse et `Control.Geocoder` pour la recherche géographique.
 - **Couche de Visualisation de Données (Dataviz)** : Assurée par **Plotly (v2.27.0)** pour le rendu des cartes arborescentes (treemaps) de répartition macroscopique et microscopique, des graphiques mensuels et des lignes de tendance (sparklines).
 - **Couche de Persistance et de Mise en Cache Client** : Exploite **IndexedDB** pour stocker localement les réponses massives des requêtes d'éléments géographiques bruts (optimisation des rechargements) et l'API **localStorage** pour mettre en cache les limites des parcs et entités administratives.
 - **Couche de Pré-calculs (Scripts Node.js)** : Transforme des bases de données de référence lourdes (fichiers CSV de l'INSEE et de l'UNESCO) en fichiers statiques légers servis au format JSON.
-

3. Fonctionnement Interne

A. Moteur Front-End

- **app.js (Orchestrateur Principal)**
 - Point d'entrée initialisant les modules MapManager, ApiService et UiRenderer.
 - Modifie le prototype natif `HTMLCanvasElement.prototype.getContext` pour injecter automatiquement le paramètre `willReadFrequently = true` lors des requêtes de contextes 2D, éliminant les ralentissements liés à la lecture récurrente des pixels par le moteur de rendu de heatmap Leaflet.
 - Lance le pré-chargement asynchrone des jeux de données statiques (INSEE, sur-fréquentation touristique, sites UNESCO et Natura 2000) dès l'ouverture de l'application.
 - Calcule dynamiquement la superficie exacte (en km²) de chaque polygone de sélection actif grâce à la méthode géodésique `L.GeometryUtil.geodesicArea`.
 - Réalise des calculs de distance spatiale (via la méthode `distanceTo` de Leaflet) pour chaque POI afin de lui associer l'arrêt de bus, la gare, l'aéroport, l'hébergement et le service d'infrastructure (parking, toilettes, borne de recharge) les plus proches.
 - Assure la gestion interactive des événements cartographiques (surbrillance de catégories, ciblage visuel de marqueurs pulsants lors de la sélection de sites protégés, rafraîchissement ou purge des caches IndexedDB/localStorage).
- **scripts/mapManager.js (Gestionnaire Cartographique)**
 - Instancie et configure la carte Leaflet sur le conteneur DOM d'identifiant `map` en ciblant par défaut le massif pyrénéen.
 - Gère la commutation entre trois fonds de carte de base : une vue Satellite (Esri World Imagery), une vue Grise épurée (Esri Canvas Light Gray) et une vue Sombre (CartoDB Dark Matter) via l'injection d'un contrôle de tuiles dédié.
 - Limite les outils de dessin interactifs (`L.Control.Draw`) à la seule génération de rectangles et de polygones simples sans auto-intersection.
 - Extrait les anneaux géométriques des polygones sous forme de tableaux de coordonnées textuelles via la méthode `getBoundsFromLayer` pour alimenter le formateur de requêtes.
 - Construit et affiche les contours des zones administratives limitrophes (voisins) en injectant un style vectoriel gris en pointillés réagissant au survol de la souris.
 - Pilote l'affichage et la suppression du marqueur de sélection spécifique doté d'une animation CSS de pulsation (`showSelectionMarker`).
- **scripts/api.js (Couche de Communication et Services)**

- Centralise l'émission des requêtes vers l'API Overpass en exploitant la méthode HTTP POST avec encodage `application/x-www-form-urlencoded`.
- Intègre un algorithme de bascule automatique (failover) parcourant séquentiellement des serveurs miroirs alternatifs (`overpass-api.de`, `overpass.private.coffee`, `overpass.kumi.systems`, `overpass.osm.ch`) si une erreur serveur ou un dépassement de quota (statuts HTTP 429, 500, 502, 503, 504) se produit.
- Implémente une protection contre le rejet des requêtes massives par Overpass (erreurs 413 Entity Too Large) en simplifiant géométriquement les anneaux de plus de 150 points par sous-échantillonnage et en retenant au maximum les 5 sous-polygones les plus vastes d'une sélection.
- Interagit avec l'API GéoGouv (`geo.api.gouv.fr`) pour charger les emprises GeoJSON des communes françaises, l'API Nominatim pour la résolution d'entités internationales, et l'API Wikidata pour extraire les descriptions multilingues, les liens de sites (sitelinks), les images hébergées sur Wikimedia Commons et l'historique démographique recensé via la propriété P1082.
- Opère un dédoublonnage géographique strict : si plusieurs nœuds, chemins ou relations partagent une clé identifiante identique (nom normalisé en minuscules combiné à sa catégorie et son type) et se situent dans un rayon géodésique de moins de 500 mètres (vérifié par la formule de Haversine), seul le premier élément est conservé.
- Analyse les métadonnées de présence numérique des entités OSM en vérifiant l'existence de clés telles que `website`, `contact:website`, `url`, `wikivoyage` ou de réseaux sociaux explicites.
- **scripts/uiRender.js (Moteur de Rendu Visuel)**
 - Génère dynamiquement l'arborescence DOM des panneaux latéraux (statistiques macroscopiques, détails de l'élément sélectionné, listes de pré-réglages de pays et subdivisions).
 - Récupère la hiérarchie des onglets administratifs cibles en consultant les configurations déclarées dans `adminLevels.js` lors d'un changement de pays actif.
 - Pilote la logique de filtrage client en maintenant des états synchronisés pour les 13 catégories principales (Tourisme, Restauration, Hébergements, Loisirs, Sport, Histoire, Nature, Commerces, Services, Transport, Santé, Bureaux, Artisanat), les sous-catégories dynamiques et les typologies de sentiers de randonnée ou cyclables.
 - Assemble et met en forme les blocs d'indicateurs HTML personnalisés : barres de remplissage horizontales calculées en pourcentage, cartes de scores de complétude des données, intensité de l'offre d'hébergement (lits par habitant) et listes de sites patrimoniaux.
 - Construit la structure de données attendue par Plotly pour instancier le treemap principal d'emprise et trois treemaps secondaires (détail des hébergements, classification SAC des sentiers piétons, segmentation du réseau cyclable).

- Exploite une fonction interne de décalage hexadécimal (`adjustColor`) pour assombrir ou éclaircir dynamiquement les couleurs de base des catégories, appliquant un rendu visuel en dégradé sur les blocs graphiques.
- **scripts/adminLevels.js (Mapping Administratif)**
 - Dictionnaire exportant la constante `ADMIN_LEVELS` qui associe les codes pays ISO 3166-1 alpha-2 à un tableau ordonné de niveaux administratifs d'OpenStreetMap (généralement les tags `admin_level` de 4 à 8) associés à leurs traductions courantes en français.

B. Scripts de Parsing Back-End (Node.js)

- **parse_insee.js (Agrégateur de Données de Capacité Touristique)**
 - Script Node.js exploitant le flux de lecture natif (`fs.createReadStream`) et le module `csv-parser` pour ingérer le jeu de données INSEE `DS_TOUR_CAP_2026_data.csv` délimité par des points-virgules.
 - Élimine les risques de double comptage en ignorant toutes les lignes dont la colonne `GEO_OBJECT` diffère de la chaîne stricte `'COM'` (niveau communal pur).
 - Isole et totalise les établissements globaux en vérifiant le croisement de la mesure `UNIT_LOC`, du classement total `_T` et des activités d'hébergement `I551`, `I552`, `I553`.
 - Dérive la capacité théorique en lits pour les hôtels (activité `I551`) en multipliant la valeur d'observation des chambres (`PLACE`) par un facteur fixe de 2.
 - Dérive la capacité en lits pour les terrains de campings (activité `I553`) en multipliant le nombre d'emplacements (`PLACE`) par un facteur fixe de 3.
 - Extrait directement la mesure de capacité de lits bruts (`BEDPLACE`) allouée aux hébergements collectifs (activité `I552`).
 - Ventile et comptabilise le nombre d'établissements selon leur nombre d'étoiles (de 1 à 5 ainsi que la mention non classé `NC`) en interceptant les lignes d'activité `I551` et `I553` dont la colonne `UNIT_LOC_RANKING` précise le rang.
 - Exporte l'arborescence finale sous forme d'un objet JSON indexé par code INSEE dans le fichier statique cible `data/insee_data.json`.
 - **parse_whc.js (Extracteur de Sites UNESCO)**
 - Lit et décode le fichier CSV source des sites du patrimoine mondial (`whc-sites-2025 (1).csv`).
 - Convertit en nombres flottants les champs `latitude` et `longitude` en écartant les enregistrements géographiquement invalides.
 - Extrait le libellé en appliquant une précedence de la colonne `name_fr` sur la colonne `name_en` et écrit le résultat tabulaire dans le fichier statique `data/whc-sites.json`.
-

4. Stratégie de Requêtage (API)

Le cœur de la collecte dynamique repose sur la génération de requêtes en langage Overpass QL transmises au serveur d'interprétation.

Structure de Requête Overpass QL

Pour une sélection géographique donnée, l'API compile une chaîne délimitant le filtre spatial via l'opérateur de polygone (`poly:"lat1 lon1 lat2 lon2..."`). L'extraction cible les trois primitives d'OSM (node, way, relation combinés via l'instruction globale `nwr`) répondant à une expression régulière validant la présence d'au moins une clé thématique principale :

```
nwr[~"^(tourism|amenity|leisure|sport|historic|natural|shop|public_transport|railway|aeroway|healthcare|office|craft)$~"."](poly:"...")
;
```

Extraction des Réseaux Linéaires (Networks)

En parallèle des entités ponctuelles, le système extrait les géométries vectorielles (out geom) des chemins et cours d'eau traversant l'emprise en interrogeant explicitement les clés `highway`, `railway`, `aerialway`, `piste:type`, `waterway`, le tag `natural=water` ainsi que les relations porteuses du tag de route spécifique (`route` incluant les valeurs `hiking`, `foot`, `bicycle`, `mtb`, `ski` ou `piste`).

Traitement Spécial des Plans d'Eau

Les surfaces aquatiques modélisées sous forme de chemins fermés géométrisés (lacs, réservoirs) sont interceptées lors du parcours de la réponse. Le script calcule dynamiquement le barycentre de leurs sommets (moyenne des latitudes et longitudes) pour les injecter dans la liste finale sous forme de POI ponctuels exploitables dans l'interface.

5. Cycle de Vie des Données

A. Pipeline de Re-catégorisation Spatiale

La méthode de transformation `detectCategoryAndType` ingère les tags bruts d'un élément OSM pour lui affecter une catégorie macroscopique normalisée. L'algorithme applique une règle stricte de surcharge d'hébergement : si la clé `tourism` contient une valeur identifiée

comme un lieu de séjour (hotel, motel, hostel, guest_house, bed_and_breakfast, apartment, holiday_flat, chalet, camp_site, caravan_site, camp_pitch, alpine_hut, wilderness_hut, shelter), la catégorie retournée bascule automatiquement sur accommodation au lieu de tourism.

B. Schéma d'Enrichissement Interne d'un POI

Chaque objet POI finalisé et stocké en mémoire par le contrôleur expose le modèle enrichi suivant :

- **Identité et Géométrie** : id (identifiant unique OSM), lat, lng, center (centroïde pré-calculé), name, category (catégorie macro) et type (valeur de tag fine).
- **Présence Numérique (digital)** : Objet imbriqué exposant les booléens hasWebsite, hasSocialMedia, hasWikivoyage, hasPhotos (déduit de la présence de tags d'images ou de clés Wikimedia Commons/Mapillary) et le compteur de langues de traduction Wikidata.
- **Métadonnées de Contribution (osmMetadata)** : Inclut timestamp (horodatage de la dernière modification), version (itération de l'objet) et user (auteur de la révision).
- **Propriétés Spatiales d'Accessibilité (calculées dans app.js)** :
 - nearestBusStopDist / nearestTrainStationDist / nearestAirportDist : Distances en mètres vers le point d'accès de transport le plus proche.
 - nearestRoadDist / nearestHikingDist / nearestCyclingDist : Distances géométriques minimales vers les segments polygones d'infrastructures de déplacement.
 - nearestWhcDist / isInWhcSite : Distance vers un site classé par l'UNESCO et drapeau booléen d'inclusion si la distance est inférieure ou égale à 1000 mètres.

C. Mise en Cache et Verrouillage de Concurrency

Afin d'éviter la saturation des navigateurs et des serveurs distants, le flux de données intègre deux mécanismes de contrôle stricts :

- **Verrouillage Mémoire (_pendingFetches)** : Toute requête déclenchée sur une zone génère une signature unique basée sur les coordonnées de son emprise. Cette clé est insérée dans une instance globale de type Map associée à la promesse de

résolution en cours. Si l'application ou l'utilisateur tente d'émettre une requête identique en parallèle, l'interface intercepte l'appel et se greffe directement sur la promesse existante en attente.

- **Persistance Validée** : Les données complètes de la zone sont enregistrées dans un conteneur IndexedDB estampillé d'un horodatage interne. Lors d'un clic ultérieur sur la même zone, si l'horodatage en cache a moins d'une heure, la phase réseau est entièrement contournée au profit d'une lecture locale immédiate.
-

6. Guide de Reprise

Pour assurer le bon fonctionnement de l'environnement de développement et étendre l'application, les étapes suivantes doivent être respectées :

1. **Installation des Dépendances Locales** : Le sous-dossier contenant les scripts de pré-traitement nécessite l'initialisation de l'environnement Node.js et l'installation du parseur CSV déclaré dans les dépendances :

```
npm install csv-parser
```

Mise à Jour des Bases Référentielles : En cas de publication d'un nouveau jeu de données par l'INSEE ou l'UNESCO, les fichiers bruts doivent être déposés dans leurs dossiers respectifs (ex: ./ZIP INSEE lits/) avant d'exécuter les parseurs en ligne de commande (node scripts/parse_insee.js et node parse_whc.js) pour mettre à jour les fichiers statiques servis au client.

Ajout d'une Nouvelle Catégorie de Filtrage : Pour intégrer une nouvelle thématique OSM, il convient d'ajouter la clé OSM correspondante dans l'expression régulière globale de la méthode `_overpassFetch` du fichier `api.js`, d'étendre la chaîne de priorisation conditionnelle dans la méthode `detectCategoryAndType` de ce même fichier, puis de déclarer l'identifiant et le libellé de la catégorie dans le tableau d'état interne du constructeur de la classe `UiRenderer`.

Invalidation Manuelle du Cache : En cas de modification structurelle du format des objets mis en cache dans le navigateur,

incrémentez la chaîne interne de versionnement de cache (définie sur v4 dans le constructeur de ApiService) pour forcer l'invalidation automatique des anciens stockages chez les utilisateurs finaux.